Week 5 Session 1

Week 5, Session 1 — From Concepts to Context: Agents in the Wild

IDSC 4101: Agentic AI for Business Applications

Session type: Condensed concept review + extended business context + tool orientation **Tools introduced:** n8n (preview), Voiceflow (preview) — hands-on build in Session 2 **Builds on:** Week 4 Sessions 1 & 2 (agent anatomy, failure modes, guardrails, human-in-the-loop)

SLIDE DECK

SLIDE 1 — Title Slide

Title: Agents in the Wild **Subtitle:** From Blueprint to Build — What Real Deployments Look Like **Course:** IDSC 4101: Agentic AI for Business Applications | Week 5, Session 1

Speaker Notes: Last week you built an agent on paper and stress-tested it. You named the failure modes, designed guardrails, and updated your blueprints. Today we use that foundation to go further — and then further still in Session 2, where you'll actually build a working agent. This session does two things. First, it closes out the conceptual arc: I want to make sure the mental model from Week 4 is solid, because Session 2 moves fast and you need the concepts to make sense of what the tools are doing. Second, it zooms out to the business landscape: what do real agent deployments actually look like across industries? What are companies actually building, what is working, and what is still being figured out? By the end of this session you'll understand where agents are being deployed today, why those use cases were chosen first, and what you should expect when you pick up n8n or Voiceflow in Session 2. Let's go.

SLIDE 2 — Where We Left Off

Title: Week 4 in 90 Seconds

Bullets: - Generative AI responds. Agentic AI acts — plans, uses tools, loops, hands off. - Four components: goal, tools, memory, reasoning loop - Four design patterns: reflection, tool use, planning, handoff - Four failure modes: goal misspecification, tool misuse, compounding errors, reward hacking - Human-in-the-loop is a spectrum (1–5), not a binary - Guardrails are business decisions, not technical ones

The one thing to carry into today: *An agent is only as good as the goal you gave it and the guardrails you wrapped around it.*

Speaker Notes: Quick consolidation before we move. I'm not going to re-teach Week 4 — you have your blueprints and your failure mode analyses. What I want to do is make sure the key ideas are active in your working memory before we build. The one-line summary is on the slide: goal quality and guardrail quality determine agent quality. Everything else is implementation. Keep that framing as we look at real deployments — you'll see it show up in every case.

SLIDE 3 — Why These Use Cases Came First

Title: Not Every Business Problem Gets an Agent First — Here's Why

The three criteria for early agent deployment:

1. High volume, structured task The task happens many times in a predictable pattern. There's enough repetition that the cost of designing the agent is recovered quickly. *Example:* Answering the same 50 customer questions, every day, across thousands of customers.

2. Low stakes per instance, high stakes in aggregate Each individual action has limited downside if wrong. But at scale, getting it right consistently matters enormously. *Example:* Routing a support ticket to the wrong team = minor delay. Misrouting 10,000 tickets per week = significant operational cost.

3. Clear, verifiable success criteria You can tell whether the agent succeeded without a human reading every output. The feedback loop is short. *Example:* Did the ticket get resolved? Did the customer escalate? Did the refund go through? Measurable.

Speaker Notes: This framework predicts which use cases organizations tackle first — and it explains a lot of the pattern you'll see when we look at real deployments. Customer service, invoice processing, meeting scheduling, research summarization — all three criteria. High-stakes, hard-to-verify tasks — like drafting legal documents, making hiring decisions, executing financial trades — are much later in the adoption curve, and they're almost always at position 4 or 5 on the human-in-the-loop spectrum even when deployed. The question to ask about any proposed agent deployment is: does this meet all three criteria? If it doesn't, you're probably not in the early-adopter use case bucket — which means the risk profile is higher and the return on investment is less certain.

SLIDE 4 — Real Deployments by Business Function

Title: Where Agents Are Being Deployed Right Now

Table:

Function	Agent type	What it does	Why it met the three criteria
Customer service	Conversational agent	Answers FAQs, routes issues, processes returns	High volume, each mistake is recoverable, CSAT is measurable
Sales	Task agent (SDR)	Researches prospects, drafts outreach, logs to CRM	Repetitive research task, low cost per bad draft, pipeline metrics are clear
Finance / accounting	Process automation agent	Matches invoices to POs, flags exceptions, routes approvals	Highly structured, dollar-value thresholds define success easily
HR	Conversational + process agent	Onboarding Q&A, benefits navigation, PTO requests	Structured policy questions, response accuracy is verifiable

Function	Agent type	What it does	Why it met the three criteria
Marketing	Task agent	Generates content variants, A/B test setups, SEO briefs	Output quality is A/B testable, high content volume
Legal / compliance	Research agent (supervised)	Summarizes case law, drafts clause options	Volume is high, but stakes per output are high — always position 4–5

Speaker Notes: Walk through this table and notice the pattern. Customer service and sales are the two most mature areas — both have been running agent pilots for two to three years now. Finance and HR are catching up fast, primarily because the tasks are so structured and the success criteria are so clear. Marketing is interesting because the evaluation is essentially always A/B testing — you find out quickly if the AI-generated variant performs, and the cost of a bad variant is minimal. Legal and compliance are the outliers: very active in research and drafting, but almost never deployed at full autonomy, because the cost of a wrong output is too high. When someone shows you an AI legal product, ask immediately: what is the human review step? If they say there isn't one, that should concern you.

SLIDE 5 — Tool Landscape: What Gets Used in the Real World

Title: The Agent Tool Stack — What You'll Actually Encounter

Three layers:

Layer 1: Agent frameworks (what builders use) - n8n — open-source workflow automation; connects apps and APIs; agents as node chains; strong for process automation agents - Voiceflow — conversational agent builder; drag-and-drop dialogue design; strong for customer-facing chatbots - LangChain / LangGraph — developer-facing; Python-based; most flexible but requires coding - Microsoft Copilot Studio, Salesforce Agentforce — enterprise platforms; bolt onto existing enterprise software stacks

Layer 2: The brain (LLM powering the reasoning) - GPT-4o, Gemini 1.5 Pro, Claude 3 — the models that handle the think step in the reasoning loop - Typically called via API; you don't see them, but they're there

Layer 3: The tools the agent uses - Web search APIs (Serper, Bing, Perplexity) - Business system APIs (Salesforce, HubSpot, Zendesk, SAP) - File readers (PDF parsers, spreadsheet connectors) - Communication tools (Gmail, Slack, Teams APIs)

Speaker Notes: You're going to use n8n and Voiceflow in Session 2, so let's be clear about where they fit. Both are Layer 1 tools — they're the environment where you design the agent's logic. Neither one is the AI. The AI is the LLM you connect to, which does the reasoning. n8n and Voiceflow are the scaffolding: they define the workflow, connect the tools, and manage the inputs and outputs. Think of them as the skeleton; the LLM is the brain. This distinction matters because a common mistake is to blame the "AI tool" when an agent behaves badly, when actually the problem is in the workflow design — the goal was wrong, the tool connection was misconfigured, or there was no error handling. The framework is neutral. It builds what you tell it to build. What you build is your decision.

SLIDE 6 — n8n vs. Voiceflow: Choosing the Right Tool

Title: n8n or Voiceflow? Use This to Decide.

Side-by-side:

	n8n	Voiceflow
Best for	Process automation agents (structured workflows)	Conversational agents (dialogue-based)
Mental model	Flowchart of steps and decisions	Script of conversations with branches
Typical use case	“Every morning, pull new leads from CRM, research each one, draft outreach, log results”	“When a customer messages us, understand their issue, answer if we can, escalate if we can’t”
Trigger	Event or schedule (a form is submitted, it’s 9am, a new row appears in a spreadsheet)	A user message (a customer types or speaks something)
Output	Data, files, API calls, database writes	A response in a chat interface
Guardrail style	If/else conditions, error branches, approval nodes	Intent scoping, fallback responses, escalation nodes

Which to use for Session 2: - Building a customer FAQ bot → Voiceflow - Building a research or automation agent → n8n - Building a meeting scheduler → either (use whichever your team finds more intuitive)

Speaker Notes: In Session 2, your team will choose one. I’m not prescribing which — but I am recommending that you make the choice based on the use case you have in mind, not on which interface looks friendlier. If your agent is fundamentally about having a conversation — a user types something, the agent responds, the user replies — Voiceflow. If your agent is fundamentally about doing a sequence of tasks without a user in the loop — fetching, processing, routing, logging — n8n. Most student builds fall clearly into one category. If you’re genuinely on the fence, go with Voiceflow for a first build — the visual dialogue flow is more forgiving of mistakes for a 75-minute build sprint. —

SLIDE 7 — What Session 2 Looks Like (Build Sprint Preview)

Title: What You’re Building in Session 2

The task: Build a working agent using n8n or Voiceflow. Your agent must: 1. Receive an input (a user message, a trigger event, or a data source) 2. Process it using at least one AI reasoning step (the LLM call) 3. Produce a useful output (a response, a routed action, a logged result) 4. Handle at least one failure or edge case gracefully (unknown input, no result, escalation)

Suggested builds (you can propose your own): - A customer service agent for a fictional business: answers product questions, escalates complaints - A meeting scheduler agent: takes a natural language request and outputs a structured calendar entry - A research agent: takes a topic, searches for information, returns a structured summary

What you’ll demo: At the end of Session 2, your team demos your agent to one other team. 5 minutes: show the happy path, then deliberately trigger the edge case you designed for.

Speaker Notes: The build sprint is the core deliverable of Week 5. Session 1 is the run-up. I want you going into Session 2 with a clear idea of what you’re building — ideally a use case you’ve already thought about, because the 75 minutes will go faster than you expect. The requirement to handle an edge case is intentional. Every team that only demos the happy path builds an agent that has never been tested. The edge case demo forces you to design for failure before you present. That’s a habit worth building. After the demo, your team writes Reflection Post 3 — 200 to 300 words on what your agent does and what you decided it should NOT be allowed to do autonomously. That assignment is a direct extension of the failure mode thinking from Week 4. Don’t leave Session 2 without that decision made.

SLIDE 8 — The Agent You'll Build Is a Slice of Something Real

Title: Your Build Sprint Agent in Real-World Context

The customer service agent pattern (most common student build):

Real deployment comparison:

Your build	Enterprise equivalent
FAQ bot for a fictional coffee brand	Intercom AI, Zendesk AI, Freshdesk Freddy
Answers 5–10 product questions	Handles 40–60% of tier-1 support volume
Escalates when it doesn't know	Routes to human agent queue with context summary
Logs the conversation	Writes to CRM, generates CSAT trigger
Runs in Voiceflow	Runs on Zendesk or Salesforce platform

The gap between your build and the enterprise version: Scale, integrations, and months of tuning. Not logic. The logic is identical.

Speaker Notes: I want to make this point clearly because students sometimes undersell their build sprint work. The agent you're building in 75 minutes has the same fundamental architecture as the agents companies are paying significant money to deploy. You're not building a toy — you're building a proof of concept at learning speed. The gap is in the integrations (you're not connecting to a real Salesforce instance), the scale (you're not handling 10,000 conversations a day), and the tuning (you haven't iterated on the prompts 50 times based on real user inputs). The logic — receive input, reason, respond, escalate — is not simplified for students. It's the real thing. That perspective matters for how you talk about this experience when someone asks you about it in an interview.

SLIDE 9 — One More Concept Before We Build: Memory in Practice

Title: What Agents Actually Remember — and What They Don't

Three types of agent memory:

1. In-context memory (short-term) Everything in the current conversation or workflow run. The agent "knows" what happened earlier in this session. Limitation: there's a limit on how much fits in context. Long conversations or large documents can exceed it.

2. External memory (retrieved) Information stored outside the model — a knowledge base, a database, a document store. The agent retrieves what it needs when it needs it. This is RAG in agent form: the agent queries its knowledge base, gets back relevant chunks, uses them to answer. Limitation: the retrieval is only as good as the indexing. If the right document isn't in the knowledge base, the agent doesn't know it.

3. Persistent memory (cross-session) Some agents are designed to remember things across separate conversations — a user's preferences, past interactions, open issues. This is less common in simple deployments but essential for anything customer-facing that should feel continuous. Limitation: requires explicit design and raises privacy considerations.

Why this matters for your build: In Session 2, your agent will primarily use in-context memory. If you build a FAQ bot, the "knowledge" is typically provided as a system prompt or a document attached to the LLM call — that's a simple form of external memory. You won't be building cross-session memory. But you should know it exists, because in your failure mode analysis you may have noted "the agent doesn't remember previous conversations" as a risk — that's a real limitation you're designing around.

Speaker Notes: Memory is the concept students most frequently under-specify in their blueprints. “It remembers the conversation” is usually what they write — which is in-context memory, and it’s correct for most single-session agents. But it creates a subtle failure mode: if a conversation gets very long or complex, the early context starts to fall out of the window, and the agent can begin contradicting itself or losing track of earlier user preferences. For your Session 2 build, this is a theoretical concern — your demos will be short. But for the capstone, if you’re designing a customer-facing agent that handles a multi-turn support conversation, you’ll want to explicitly design the memory layer. Make a note of that now.

SLIDE 10 — Before You Go: Setting Up for Session 2

Title: Session 2 Prep — Do This Before You Leave Today

What your team should decide before Session 2:

☐ **1. Pick your use case** Decide what your agent will do. One sentence: “Our agent [does X] for [Y type of user] when [Z trigger occurs].”

☐ **2. Pick your tool** n8n (process automation) or Voiceflow (conversational). Use the decision framework from Slide 6.

☐ **3. Assign a driver** One person drives the build (shares screen or keyboard). Others navigate, test inputs, and catch errors. Rotate if you finish early.

☐ **4. Name your edge case** Before Session 2 starts, your team should know what the “expected failure” is — the input or condition your agent can’t handle gracefully. You’ll build a response for it. Deciding it in advance saves 15 minutes of in-session debate.

☐ **5. Bring your Week 4 blueprint** Your agent from Week 4 may or may not be what you build in Session 2. But your blueprint has the goal, tools, and guardrail thinking you already did. It’s useful reference material.

Speaker Notes: I’m giving you 10 minutes at the end of today’s session to make these decisions as a team. Don’t leave without consensus on 1 and 2 — those are the blockers. If a team comes into Session 2 still debating the use case, they’ll spend the first 20 minutes of the build sprint arguing instead of building. Use the time now.

SLIDE 11 — Team Prep Time + Reflection Post Reminder

Title: 10 Minutes: Team Decisions for Session 2

Reflection Post 3 (due before Week 6 Session 1):

Prompt: 200–300 words. After your Session 2 build: - What does your agent do? (One clear sentence.) - What did you decide your agent should NOT be allowed to do autonomously — and why? - What was the hardest design decision your team made?

This assignment is not a description of your build. It’s a reflection on your design decisions.

Graders are looking for: specificity (which guardrail, which trigger, which constraint?), reasoning (why that decision and not a different one?), and honesty (what would you do differently?).

Speaker Notes: Remind students that the reflection post is due before Week 6 Session 1, not immediately after Session 2. That’s intentional — you should have time to sit with the build, think about what you actually decided, and write something considered. The best reflection posts I’ve read describe a moment in the build

sprint where the team realized something wasn't going to work as planned — and explain what they changed and why. That moment of revision is the learning. If your reflection says "everything went smoothly," I'll ask follow-up questions.

SLIDE 12 — Close

Title: Today → Session 2 → Reflection

Today accomplished: - Consolidated the Week 4 framework — goal quality + guardrail quality = agent quality - Mapped the real business deployment landscape by function - Oriented to n8n and Voiceflow and how to choose between them - Previewed the Session 2 build sprint and its requirements - Teams made use case and tool decisions for Session 2

Session 2: 75-minute build sprint. You build, you demo, you reflect.

Closing thought: Every agent you encounter in your career started as a decision someone made about what the goal was and what the system was not allowed to do. You've spent two weeks making those decisions on paper. In 75 minutes, you'll make them in a system that actually runs. That shift from design to execution is where the real learning happens. See you in Session 2.

Speaker Notes: Keep the close tight. The teams need the last 10 minutes for prep. End with something concrete: "Take out your laptops or a piece of paper. You have 10 minutes. Make the five decisions on Slide 10. I'll circulate and check in." Don't let this become free-form discussion time — focused preparation is more valuable than another 10 minutes of lecture at this point in the session.

ACTIVITY PLAN

Activity: "Session 2 Prep" — Team Decision Sprint (10 minutes, end of session)

Purpose: Ensure every team enters Session 2 with a decided use case, chosen tool, assigned driver, named edge case, and their Week 4 blueprint in hand. This is a structured alignment exercise, not open brainstorming — teams should be deciding, not generating.

Team configuration: Same teams as Week 4. No regrouping needed.

What students do:

Phase	Duration	Task
Use case decision	3 min	Teams converge on a one-sentence use case. Instructor circulates — listen for vagueness and redirect toward specificity: "What exactly does it do? For whom? When?"
Tool selection	2 min	Apply the Slide 6 framework. Pick n8n or Voiceflow. If the team is split, default to Voiceflow for a first build.

Phase	Duration	Task
Driver assignment + edge case naming	3 min	Assign roles for Session 2. Name the edge case: what input will break the happy path?
Confirm blueprint is accessible	2 min	Teams check that their Week 4 blueprint (paper, photo, or shared doc) is accessible for Session 2.

What students produce: No formal submission. Before leaving, each team should be able to answer: What are you building? In which tool? Who's driving? What's your edge case?

Instructor facilitation notes:

During use case decision:

Common failure mode: teams choose a use case that sounds impressive but is too complex for 75 minutes. A multi-step research agent that searches three sources, synthesizes findings, generates a report, and emails it to stakeholders is a Week 6 capstone idea, not a build sprint. Redirect toward simplicity: "What's the core loop? What's the minimum useful version?" A customer service agent that answers 5 questions and escalates is genuinely achievable in 75 minutes. A fully integrated sales pipeline agent is not.

Also watch for teams that pick the exact same use case as their Week 4 blueprint — which is fine, but prompt them: "Is this achievable in 75 minutes? What would you cut if you had to build only the essential version?" The blueprint is a design document; the build sprint requires a scoped, buildable version.

During tool selection:

Most teams will default to Voiceflow because it's more visually familiar. That's a reasonable choice for most use cases. Redirect to n8n only if the use case is clearly process-oriented (no user conversation, just triggered automation). Mixed-type builds — where there's both a conversation and a downstream automation — should start with the conversational layer in Voiceflow and treat the automation as a stretch goal.

During edge case naming:

Many teams will say "we'll figure it out in Session 2." Push back gently: "Name one input that your agent won't know how to handle. Just one." This takes 30 seconds and prevents a scenario where the team gets to the demo and realizes they have no failure handling. Acceptable edge cases: a question outside the agent's knowledge base, an ambiguous request with multiple valid interpretations, an input in a language the agent wasn't designed for, a request the agent is explicitly not authorized to fulfill.

Pacing note:

10 minutes is tight but sufficient if you start it cleanly. Announce it as a structured activity with a timer, not as "take some time to discuss." Teams that are given unstructured time will drift into feature planning and miss the decisions. Set a visible timer if possible.